Ist-Daten v2: jeder Halt mit Soll- und Ist-Zeit

Merkmal	Befund
Herkunft	Geschäftsstelle SKI (SBB AG) im Auftrag des BAV, opentransportdata.swiss
Zugang und Lizenz	Monatsarchiv als Zip, frei ohne Login, Quellenangabe Pflicht
Abdeckung	v2 ab Juli 2025, 1.1. bis 31.3.2026 vollständig (90 Tage)
Format	CSV mit Trennzeichen «;», UTF-8, 22 Spalten, eine Datei pro Tag
Grösse	420 bis 650 MB pro Tag, ca. 2,4 Mio. Zeilen (ganze Schweiz)
Zeitformat	Lokalzeit ohne Zeitzone, Sommerzeit am 29.3.2026 im Zeitraum

```
Wichtige Spalten: BETRIEBSTAG, PRODUKT_ID, LINIEN_TEXT, BPUIC, ANKUNFTSZEIT, AN_PROGNOSE,
AN_PROGNOSE_STATUS, FAELLT_AUS_TF, DURCHFAHRT_TF
```

Ingestion: ein Tag nach dem anderen, wiederholbar



Idempotent

Geladene Tage werden übersprungen, ein neuer Lauf erzeugt keine Duplikate.

Protokolliert

Pro Tag Zeilen vor und nach dem Filter, Dauer und Fehler in Logfile und Tabelle ingest_log.

Zeitzonen-sicher

Alle Zeiten als TIMESTAMPTZ (ISO 8601 mit Zeitzone), Sommerzeit korrekt.

```
python src/ingest_istdaten.py --start 2026-01-01 --end 2026-03-31 --stops
data/stammdaten/haltestellen_ostschweiz.csv
```

Von 215 Mio. auf 17 Mio. Zeilen

215 Mio.

Zeilen roh, 90 Tage, ganze Schweiz

2587

Haltestellen in SG, TG, AR und AI

17,07 Mio.

Halte in der Datenbank (Bahn 1,75, Bus 15,32)

Beim Laden bereinigt

- Exakte Duplikate entfernt, gemessener Wert vor Prognose
- 9-stellige BPUIC auf die 7-stellige Haltestelle gekürzt
- Lokalzeit in TIMESTAMPTZ (Europe/Zurich) umgewandelt

Offengelegt für Paket C

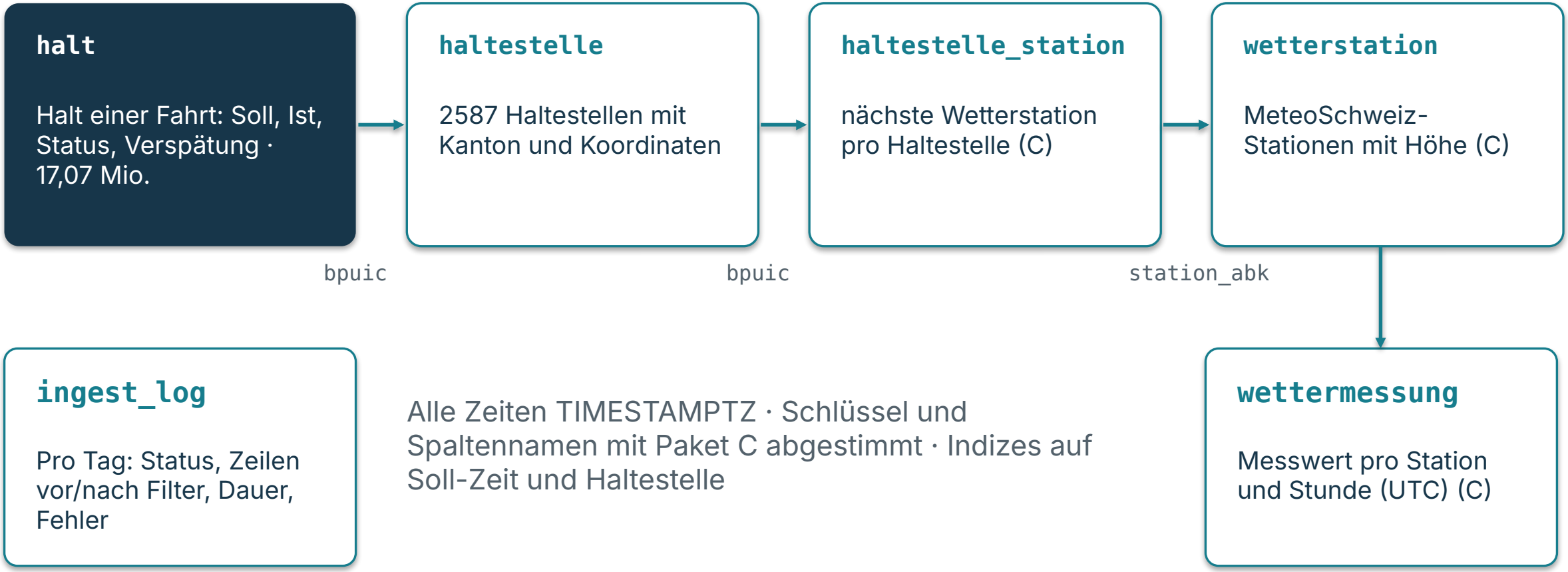
- Ankunft echt gemessen (REAL): Bahn 89,7 %, Bus 65,9 %
- Unplausible Ist-Zeiten (z. B. genau 1 Tag daneben) bleiben roh
- Status jeder Zeit (REAL, PROGNOSE, GESCHAETZT) wird mitgeführt

Storage: DuckDB – analytisch, eine Datei, kein Server

Kriterium	DuckDB	SQLite	PostgreSQL	MongoDB
Analytische Abfragen (Joins, GROUP BY)	sehr gut	mässig	gut	mässig
CSV streamend einlesen	sehr gut	mässig	gut	mässig
Zeitzonen (TIMESTAMPTZ)	gut	schwach	sehr gut	mässig
Skalierung auf mehr Kantone	gut	schwach	sehr gut	gut
Setup für die Gruppe	sehr gering	sehr gering	hoch (Server)	hoch (Server)
Viele gleichzeitige Nutzer	mässig	mässig	sehr gut	gut

Entscheid: DuckDB für Teil 1. Für das Dashboard in Teil 2 ist ein Wechsel auf PostgreSQL möglich, weil das Schema Standard-SQL ist.

Datenmodell: Halt und Wetter über die Haltestelle verknüpft



Alle Zeiten TIMESTAMPTZ · Schlüssel und Spaltennamen mit Paket C abgestimmt · Indizes auf Soll-Zeit und Haltestelle

Skalierbarkeit: gleicher Code, längere Haltestellenliste

Umfang	Zeilen	DuckDB	Parquet	Laufzeit
1 Tag, ganze Schweiz (gemessen)	2,34 Mio.	375 MB	ca. 45 MB	43 s
90 Tage, Ostschweiz (gemessen)	**17,07 Mio.**	**2,6 GB**	**305 MB**	**ca. 16 Min.**
90 Tage, ganze Schweiz (hochgerechnet)	ca. 210 Mio.	ca. 34 GB	ca. 4 GB	ca. 1 h

- Speicherbedarf beim Laden hängt nicht von der Dateigrösse ab, weil streamend gelesen wird.
- Mehr Kantone: nur die Haltestellenliste erweitern, Code und Schema bleiben gleich.
- Grenze: DuckDB erlaubt nur einen schreibenden Prozess; für Teil 2 ist PostgreSQL die nächste Stufe.

Paket B: mögliche Fragen 1/2 (Anhang)

Warum nicht PostgreSQL?

Für Teil 1 brauchen wir Analyse, keinen Mehrbenutzerbetrieb. DuckDB ist schneller aufgesetzt und spaltenorientiert; das Schema ist Standard-SQL, ein Wechsel ist einfach.

Was heisst idempotent bei euch?

Jeder Tag wird in einer Transaktion gelöscht und neu eingefügt. Ein zweiter Lauf überspringt fertige Tage. Geprüft: zweiter Lauf, 90 Tage übersprungen.

Wie geht ihr mit der Sommerzeit um?

Rohzeiten sind Lokalzeit. Wir wandeln sie mit Europe/Zurich in TIMESTAMPTZ um, so stimmen Dauern über die Umstellung am 29.3.

Paket B: mögliche Fragen 2/2 (Anhang)

Wie zuverlässig sind die Ist-Zeiten?

Ankunft gemessen: Bahn 89,7 %, Bus 65,9 %, der Rest sind Prognosen oder Schätzungen. Der Status ist pro Zeit gespeichert, Paket C nimmt nur gemessene Werte.

Wie habt ihr die Haltestellen den Kantonen zugeordnet?

Koordinaten aus dem GTFS-Fahrplan, dann Punkt-in-Polygon mit den Kantonsgrenzen von swisstopo. Ergebnis: 2587 Haltestellen.

Warum liegen die Rohdaten nicht im Repo?

Rund 50 GB ungefiltert. Im Repo liegt ein Beispieltag, das Skript lädt alles mit einem Befehl nach. Die gefilterten Daten liegen als Parquet auf OneDrive.